Data Structures

BS (CS) _Fall_2025

Lab_03 Task

Learning Objectives:

1. 2D and 3D arrays

Task 0: Templates with Arrays (Insert, Delete, Display)

Problem Statement:

Create a template class Array3D<T> that stores elements in a 3D dynamic array.
The class should support basic insert, delete, and display operations.

Features to Implement:

Constructor

- Takes 3 dimensions (x, y, z) as input.
- Allocates a dynamic 3D array of that size.
- Initializes all values to 0 (or T{} default constructor).

insert(int x, int y, int z, T value)

- Insert value at position [x][y][z].
- If indices are out of range, show an error message.

deleteAt(int x, int y, int z)

- Delete element at [x][y][z] by resetting it back to 0 (or T{}).
- If indices are out of range, show an error message.

printArray()

- Print the entire 3D array in a readable format.

Google Test Cases (GTest)

1. Basic Array Tests (T = int)
    - Create Array3D<int> arr(2, 2, 2).
    - Verify all elements initialized to 0.
    - Insert 42 at (0,1,1) → element at (0,1,1) should be 42.
    - Insert value at out-of-range index (e.g., (3,0,0)) → should show error.
    - Delete element at (0,1,1) → element should reset to 0.
    - Delete element at out-of-range index (2,2,2) → should show error.
    - Print array after some insertions → output should contain inserted values.
2. Different Data Type Tests
    - Array3D<double>
    - Create Array3D<double> arr(2,2,1).
    - Insert 3.14 at (1,0,0).
    - Delete (1,0,0) → value resets to 0.0.

- Array3D<char>
- Create Array3D<char> arr(2,2,2).
- Insert 'A' at (0,0,1).
- Delete (0,0,1) → value resets to '\0'.

Task 1: City Transport System

Problem Statement

A city transport authority wants to model the bus travel times between different stops.

We represent this using a 2D adjacency matrix:

- Each row represents the starting stop.
- Each column represents the destination stop.
- The entry A[i][j] tells us the time (in minutes) to travel from stop i to stop j directly.
- If there is no direct bus from stop i to stop j, we store a large constant INF (meaning "no connection").
- The diagonal entries A[i][i] are always 0 (it takes 0 minutes to stay at the same stop).

Example

Suppose we have 3 stops:

- Stop 0 = Airport
- Stop 1 = Bus Terminal
- Stop 2 = University

Matrix A:

```
      To
      0    1    2
From
0   [ 0   15  INF ]
1   [INF   0   10 ]
2   [  6  INF   0 ]
```

Interpretation:

- From Airport -> Bus Terminal = 15 minutes (A[0][1] = 15).
- From Bus Terminal -> University = 10 minutes (A[1][2] = 10).
- From Airport -> University = INF (no direct bus).

If we allow one transfer at stop 1, then Airport -> University becomes:

Airport -> Bus Terminal (15) + Bus Terminal -> University (10) = 25

Note :

Implement a **template class** `TransportMatrix<T>` using a **dynamic 2D array** (`new`/`delete`).

Features to Implement

Constructor

- Takes number of stops (rows/cols).
- Initializes all entries to INF.
- Sets diagonal to 0.

setTime(from, to, time)

- Update the travel time between two stops.

getTime(from, to)

- Return the travel time.

reverseRoutes()

- Return a new matrix where all routes are reversed.

withOneTransfer(int middleStop)

- For each pair (i, j), check if going via middleStop is shorter than going directly
- Update with the shorter option.

printRoutes()

- Print the full adjacency matrix (show "INF" for no bus).

GTEST for City Transport system:

```
const int INF = 1000000; // same as in implementation

// Test constructor initialization
TEST(TransportMatrixTest, ConstructorInitializesCorrectly_Int) {
    TransportMatrix<int> tm(3);
    EXPECT_EQ(tm.getTime(0,0), 0);
    EXPECT_EQ(tm.getTime(1,1), 0);
    EXPECT_EQ(tm.getTime(2,2), 0);
    EXPECT_EQ(tm.getTime(0,1), INF);
    EXPECT_EQ(tm.getTime(2,0), INF);
}

// Test setting and getting values
TEST(TransportMatrixTest, SetAndGetTime_Int) {
```

```
    TransportMatrix<int> tm(3);
    tm.setTime(0,1,15);
    tm.setTime(1,2,10);
    tm.setTime(2,0,6);
    EXPECT_EQ(tm.getTime(0,1), 15);
    EXPECT_EQ(tm.getTime(1,2), 10);
    EXPECT_EQ(tm.getTime(2,0), 6);
}

// Test reverse routes
TEST(TransportMatrixTest, ReverseRoutes_Int) {
    TransportMatrix<int> tm(3);
    tm.setTime(0,1,15);
    tm.setTime(1,2,10);

    TransportMatrix<int> reversed = tm.reverseRoutes();
    EXPECT_EQ(reversed.getTime(1,0), 15);
    EXPECT_EQ(reversed.getTime(2,1), 10);
    EXPECT_EQ(reversed.getTime(0,1), INF);
    EXPECT_EQ(reversed.getTime(1,2), INF);
}

// Test one transfer
TEST(TransportMatrixTest, WithOneTransfer_Int) {
    TransportMatrix<int> tm(3);
    tm.setTime(0,1,15);
    tm.setTime(1,2,10);
    tm.setTime(0,2,INF);

    tm.withOneTransfer(1);
    EXPECT_EQ(tm.getTime(0,2), 25);  // via stop 1
}

// Now test with double type
TEST(TransportMatrixTest, ConstructorInitializesCorrectly_Double) {
    TransportMatrix<double> tm(2);
    EXPECT_EQ(tm.getTime(0,0), 0.0);
    EXPECT_EQ(tm.getTime(1,1), 0.0);
    EXPECT_EQ(tm.getTime(0,1), INF);
}

TEST(TransportMatrixTest, SetAndGetTime_Double) {
    TransportMatrix<double> tm(2);
    tm.setTime(0,1,12.5);
    EXPECT_DOUBLE_EQ(tm.getTime(0,1), 12.5);
```

```
}

TEST(TransportMatrixTest, ReverseRoutes_Double) {
    TransportMatrix<double> tm(2);
    tm.setTime(0,1,7.5);

    TransportMatrix<double> reversed = tm.reverseRoutes();
    EXPECT_DOUBLE_EQ(reversed.getTime(1,0), 7.5);
    EXPECT_EQ(reversed.getTime(0,1), INF);
}

TEST(TransportMatrixTest, WithOneTransfer_Double) {
    TransportMatrix<double> tm(3);
    tm.setTime(0,1,5.5);
    tm.setTime(1,2,4.5);
    tm.setTime(0,2,INF);

    tm.withOneTransfer(1);
    EXPECT_DOUBLE_EQ(tm.getTime(0,2), 10.0);
}
```

Task 2: Hospital Patient Vitals Analytics

Problem Statement

A hospital records a vital metric (e.g., average daily heart rate or temperature) across multiple days, multiple wards, and multiple patients.

Implement a template class VitalsAnalytics<T> that stores readings in a 3D dynamic array addressed as:

`readings[wards][patient][day]`

- Each day contains multiple wards.
- Each ward has readings for multiple patients.
- You must allocate, access, and free this structure using raw pointers (T***) with new[] / delete[].
- No STL containers for core storage.

Functional Requirements

Constructor / Destructor

VitalsAnalytics(int days, int wards, int patients)

- Dynamically allocate T*** and initialize all entries to T{}.
- Properly free all memory in the destructor (reverse order).

setReading(day, ward, patient, value)

- Store a reading at the specified indices.

getReading(day, ward, patient)

- Return the reading at the specified indices.

patientTrend(ward, patient Id)

- Return the readings of all the day for a single patient, as a new dynamically allocated 1D array (array length = total days of the patient).
- Caller owns the returned pointer and must delete[] it.

wardPeakDay(wardId)

- Return the day index on which this ward's total reading (sum over patients) is maximum.
- Ties -> return the smallest day index.

comparePatients(ward, p1, p2)

- Compare two patients by their overall average across all days and wards.
- Return 1 if p1 > p2, -1 if p2 > p1, 0 if equal.

printReport()

- Print a concise summary:
- For each patient: overall average.
- For each ward: peak day.

```
// ---------- Allocation & Initialization ----------
TEST(VitalsAnalyticsTest_Int, HandlesAllocationAndSetGet) {
    VitalsAnalytics<int> vitals(2, 2, 2); // 2 wards, 2 patients, 2 days
    vitals.setValue(0, 0, 0, 75);
    vitals.setValue(0, 0, 1, 80);

    EXPECT_EQ(vitals.getValue(0, 0, 0), 75);
    EXPECT_EQ(vitals.getValue(0, 0, 1), 80);
}

TEST(VitalsAnalyticsTest_Double, HandlesAllocationAndSetGet) {
    VitalsAnalytics<double> vitals(1, 2, 2);
```

```
    vitals.setValue(0, 1, 0, 98.6);
    vitals.setValue(0, 1, 1, 99.1);

    EXPECT_NEAR(vitals.getValue(0, 1, 0), 98.6, 1e-6);
    EXPECT_NEAR(vitals.getValue(0, 1, 1), 99.1, 1e-6);
}

// ---------- Patient Trend ----------
TEST(VitalsAnalyticsTest_Int, PatientTrendWorks) {
    VitalsAnalytics<int> vitals(1, 1, 3); // 1 ward, 1 patient, 3 days
    vitals.setValue(0, 0, 0, 70);
    vitals.setValue(0, 0, 1, 75);
    vitals.setValue(0, 0, 2, 80);

    int* trend = vitals.patientTrend(0, 0);
    EXPECT_EQ(trend[0], 70);
    EXPECT_EQ(trend[1], 75);
    EXPECT_EQ(trend[2], 80);
    delete[] trend;
}

TEST(VitalsAnalyticsTest_Double, PatientTrendWorks) {
    VitalsAnalytics<double> vitals(1, 1, 2);
    vitals.setValue(0, 0, 0, 98.4);
    vitals.setValue(0, 0, 1, 99.2);

    double* trend = vitals.patientTrend(0, 0);
    EXPECT_NEAR(trend[0], 98.4, 1e-6);
    EXPECT_NEAR(trend[1], 99.2, 1e-6);
    delete[] trend;
}

// ---------- Ward Peak Day ----------
TEST(VitalsAnalyticsTest_Int, WardPeakDayCorrect) {
    VitalsAnalytics<int> vitals(1, 2, 2);
    // Day 0 totals = 100, Day 1 totals = 150
    vitals.setValue(0, 0, 0, 40);
    vitals.setValue(0, 1, 0, 60);
    vitals.setValue(0, 0, 1, 70);
    vitals.setValue(0, 1, 1, 80);

    EXPECT_EQ(vitals.wardPeakDay(0), 1);  // Peak on day 1
}

// ---------- Compare Patients ----------
```

```
TEST(VitalsAnalyticsTest_Double, ComparePatientsEqual) {
    VitalsAnalytics<double> vitals(1, 2, 2);
    // Patient 0 avg = 100.0, Patient 1 avg = 100.0
    vitals.setValue(0, 0, 0, 100.0);
    vitals.setValue(0, 0, 1, 100.0);
    vitals.setValue(0, 1, 0, 100.0);
    vitals.setValue(0, 1, 1, 100.0);

    EXPECT_EQ(vitals.comparePatients(0, 0, 1), 0);  // Equal
}

TEST(VitalsAnalyticsTest_Double, ComparePatientsDifferent) {
    VitalsAnalytics<double> vitals(1, 2, 2);
    // Patient 0 avg = 110.0, Patient 1 avg = 100.0
    vitals.setValue(0, 0, 0, 110.0);
    vitals.setValue(0, 0, 1, 110.0);
    vitals.setValue(0, 1, 0, 100.0);
    vitals.setValue(0, 1, 1, 100.0);

    EXPECT_EQ(vitals.comparePatients(0, 0, 1), 1);  // Patient 0 > Patient 1
}
```